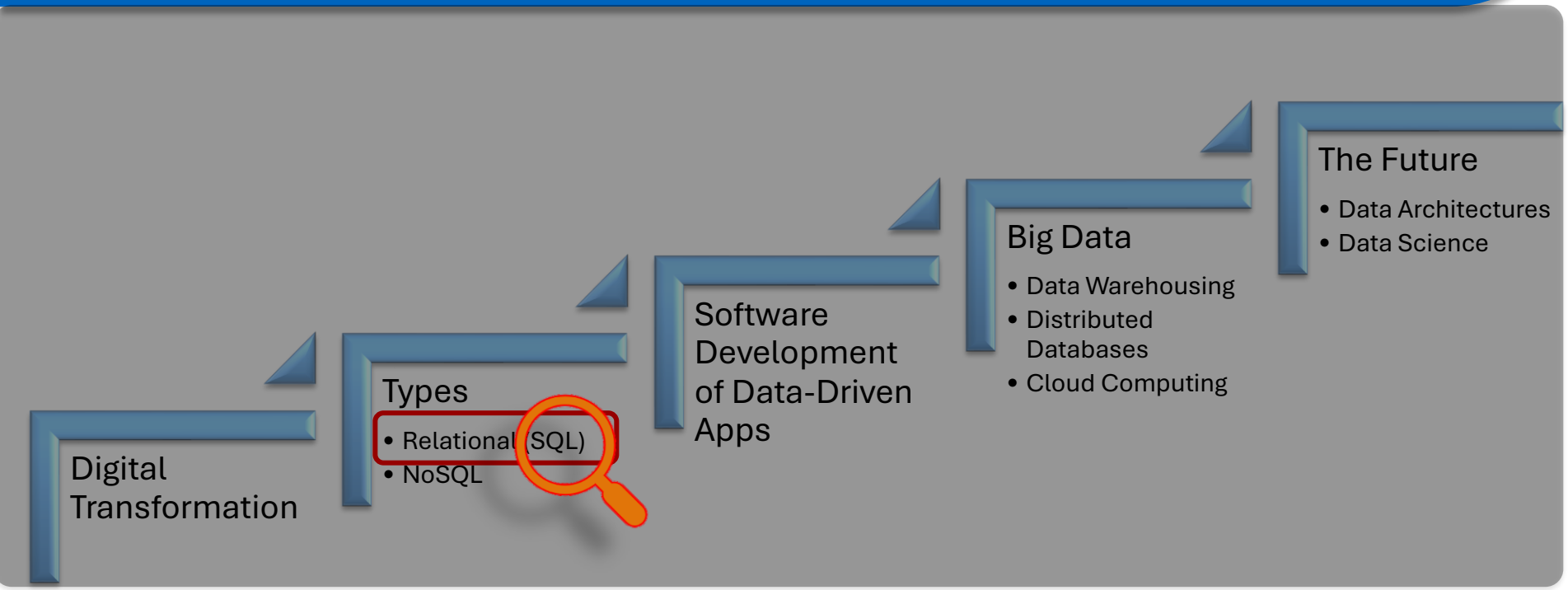
CCS1510 - Introduction to Databases

Google Classroom: **fveqiigm**

Miltos Vafiadis - mvafiadis@yorkeuropecampus.eu

Module Overview



Data Modeling	SQL Language	Techniques	Theory behind Databases
• ERD • Relational Data Model	• Queries • SQL Programming	• Normalization	• Relational Algebra

Why Databases

Databases are everywhere!

Everyday Activities

- Bank withdrawals and deposits
- Travel Reservations
- Library catalog search
- Shopping
- eGovernment

Business Applications

- CRM, ERP, HRM

Other Applications

- Multimedia
- Geographic Information Systems
- Data Warehouses
- Smart Devices / Homes

➡ Electronic storage and fast, online retrieval of data

DIGITAL
TRANSFORMATION

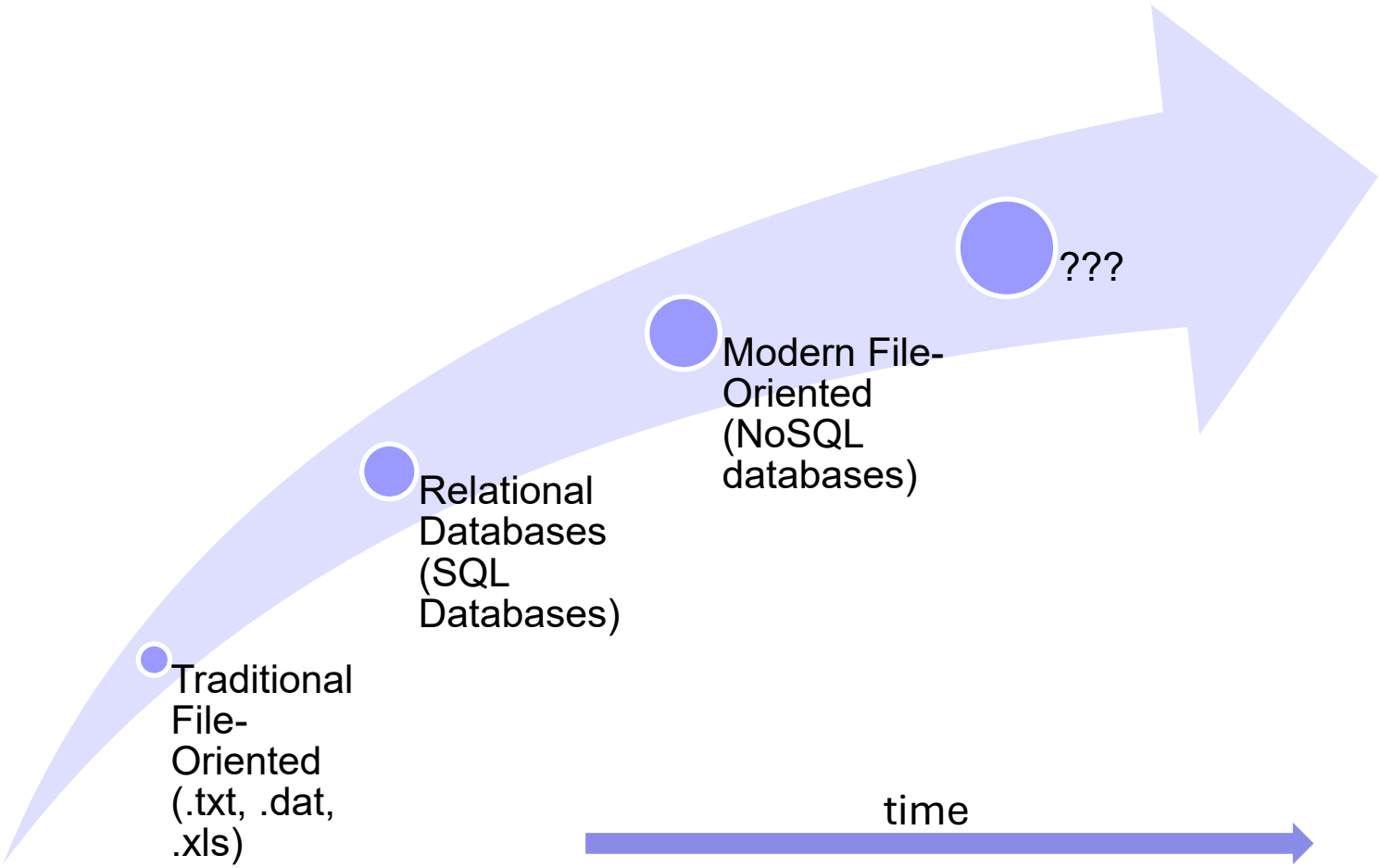
Important processes related to data are:

- Collection
- Organization
- Analysis
- Dissemination (spread / share)
- Interpretation

Organizations gather data from:

- Internal operations
- External sources
- The environment

Approaches to Data Storage



Approach #1 - “Old” File

- **Data Redundancy**: identical data stored in two or more files → multiple updates, storage waste, data inconsistency
- **Lack of Data Integration**: cannot associate related data from different files in a logical way
- **Program/data dependence**: changes in the format of a file require changes in all programs that access the file
- **Lack of flexibility**: unanticipated request for new information requires extensive programming

Approach #2 - Relational Database

A database is a ***collection of related data***

Properties of a Database

- Represents some aspect of the real world
- *A logically coherent collection* of data
  - a random assortment of data cannot be called a database
- Designed, build and populated with data *for some specific purpose*
  - Intended group of users
  - Preconceived applications

Relational Database History

1970

IBM Scientist Edgar Codd

A Relational Model of Data for Large Shared Data Banks

1979

Commercial Releases

- Oracle
- DB2, SAP Sybase, Informix

1980s-1990s

Increasingly Dominant

Efficient Queries, Transactions

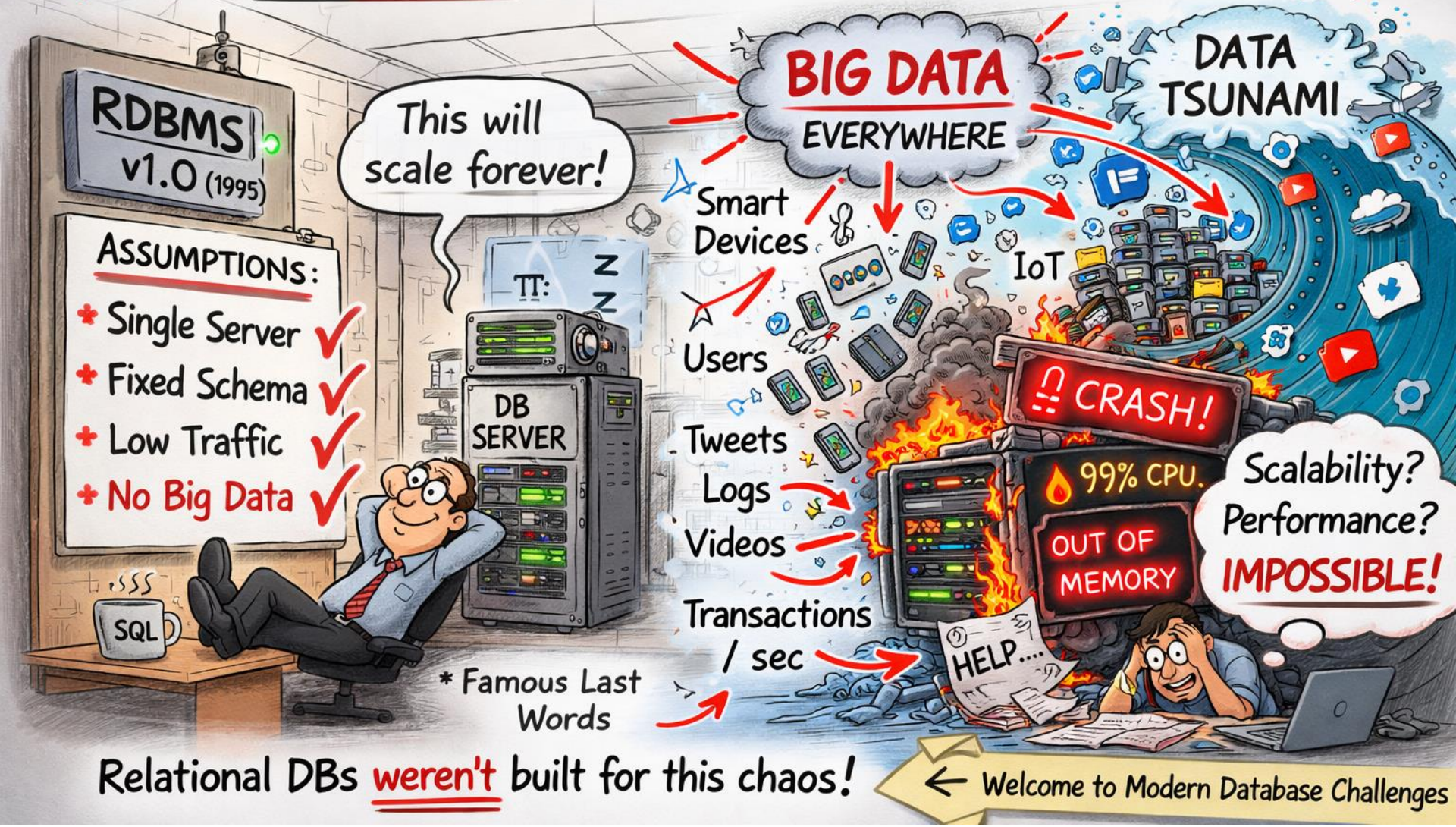
SQL was “the language of data”

Late 2000s

Still popular, but anything requiring scale → NoSQL

Relational DBs: Designed for a Simpler World...

(Run on ONE Server. Before Internet & Smart Devices. Oops.)



Relational DBs weren't built for this chaos!

← Welcome to Modern Database Challenges

Relational Database Issues

- **Big Data**
  - High Volume, Velocity, Variety
  - So massive, traditional data processing software just can't manage them

- Relational Systems were architected around **Assumptions**
  - Run on Single Server
  - Before internet gained massive popularity
  - Before "smart devices" generating tons of data

- Major Issues: **Scalability / Performance**

Approach #3 - NoSQL

- Flat files
  - Similar to the old, traditional file approach
  - Removes some of the great relational DBMS advantages

- But…
  - Easy to scale
  - Used for Distributed data stored with huge data storage needs

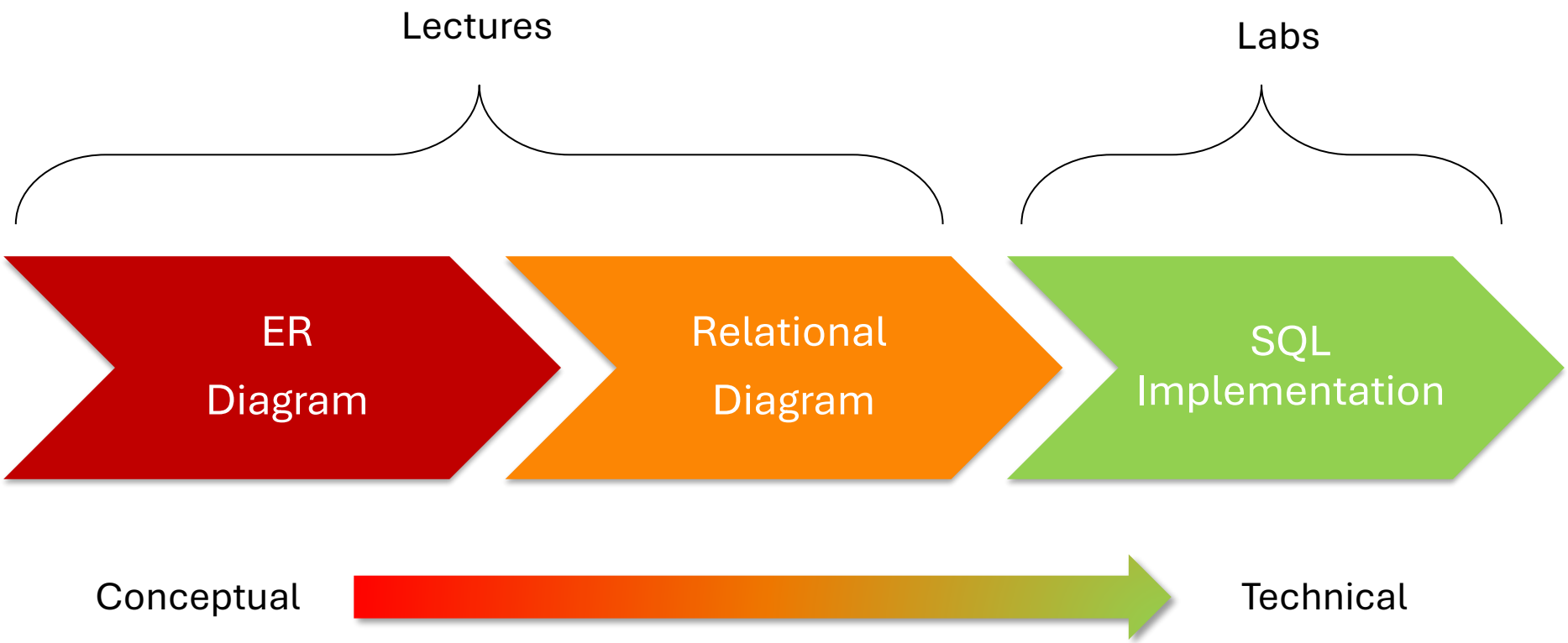
Future of Databases

- NoSQL approaches are trying to include more of the Relational advantages
- Relational approaches are trying to achieve more of the scalability and true distributed logic that NoSQL provides

This module will focus on the **Relational** approach

We will explore NoSQL and other Advanced Topics towards the end of the semester

CCS1510 Module Approach



DBMS - Definition

- ✓ A **D**ata**B**ase **M**anagement **S**ystem is a complex collection of programs that enables users to create and maintain a database

A DBMS facilitates the processes of:

- **Defining**
  - Specifying data types, structure, constraints
- **Constructing**
  - Storing the data on a medium controlled by the DBMS
- **Manipulating**
  - Querying, updating, report generation

DBMS - Types

Categorization of DBMSs is done according to:

Model

- Relational
- Object-oriented, object-relational, … etc

Number of sites

- Centralized
- Distributed
  - Homogeneous
  - Heterogeneous (federated, multi-database systems)

Purpose

- General-purpose
- Special-purpose (OLTP vs. OLAP)

DBMS - Environment

DBMS - Advantages

1. Minimize Redundancy
2. Restrict Unauthorized Access
3. Provide Multiple User Interfaces
4. Enforce Integrity Constraints
5. Provide Backup and Recovery

1. Minimize Redundancy

Store each logical piece of data in only one place in the database

- *A customer's Tax Identification Number is only stored once in the "Customer" entity, not with every Invoice*

2. Restrict Unauthorized Access

- Users should have different Security Levels
  - Some users retrieve, others retrieve and update
  - Some should only have access to a subset of data

- A Database Administrator (DBA) defines Security
  - **Authentication**
    - Who can login
  - **Authorization**
    - What data they can access

3. Provide Multiple User Interfaces

Type of Interface	Language / Tool Used	Actors
Direct Access	Query Language SQL, T-SQL, PL/SQL	Database Administrators IT Professionals Data Engineers
Programmatic Access - Traditional	ODBC, JDBC, ADO.NET, Native Drivers for Direct Database Connection	Software Developers
Programmatic Access - Modern	ORM Frameworks - Entity Framework, Hibernate, Django ORM, Sequelize, Prisma *Object mapping, type safety, database independence*	Application Developers
API-Based Access	REST APIs, GraphQL, gRPC	Full-stack Web Developers
BI & Analytics Tools	PowerBI, Tableau, Looker, Qlik	Data Analysts, Business Analysts, Decision Makers
Cloud-Native and No-code Platforms	Visual DB Builders - Airtable, Notion, Softr, SaaS - Supabase, Firebase	Business Users
System-to-System Integration	Data Warehouses, ETL Pipelines, Microservices, ML Models	Data Scientists, Data Engineers

4. Enforce Integrity Constraints

- ✓ **Same Datatype**
  - *"Date of Birth" column should have date values*
- ✓ **Correct Relationship between Entities**
  - *A "Student" belongs to one or more "Department"*
- ✓ **Uniqueness of Values**
  - *The Tax Identification Number of a Customer must be unique among all customers*

5. Provide Backup and Recovery

Include facilities for recovering from Hardware or Software failures

- ✓ Restore to the state prior to execution
- ✓ Resume from the interruption point

➢ *E.g: Failed Update*

DBMS - Self Describing Nature

A DBMS contains:

- **Catalog**
  - Definition and description of database
  - E.g. Table / column names, data types, constraints
- **Meta-data**
  - Structure of each file, type and storage format of each data item, constraints on data)

In relational DBMSs, the catalog itself is stored as tables

Users (or the system) can manipulate the catalogue using SQL

DBMS – Catalog Example

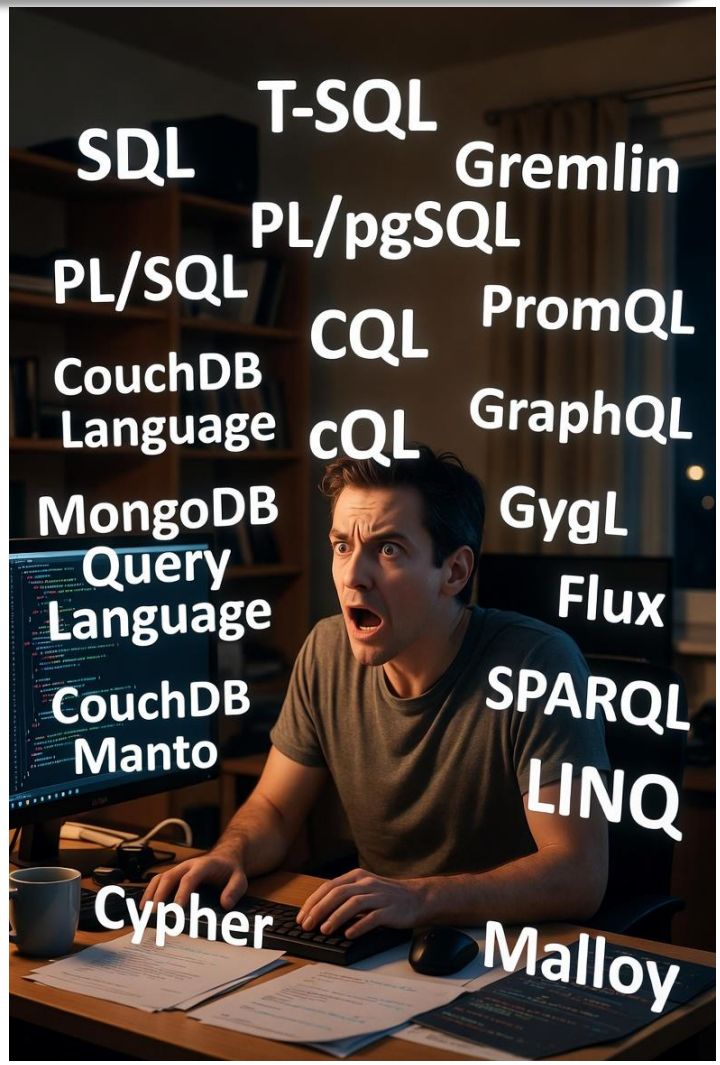
REL_AND_ATTR_CATALOG

REL_NAME	ATTR_NAME	ATTR_TYPE	MEMBER_OF_PK	MEMBER_OF_FK	FK_RELATION
EMPLOYEE	FNAME	VSTR15	no	no	
EMPLOYEE	MINIT	CHAR	no	no	
EMPLOYEE	LNAME	VSTR15	no	no	
EMPLOYEE	SSN	STR9	yes	no	
EMPLOYEE	BDATE	STR9	no	no	
EMPLOYEE	ADDRESS	VSTR30	no	no	
EMPLOYEE	SEX	CHAR	no	no	
EMPLOYEE	SALARY	INTEGER	no	no	
EMPLOYEE	SUPERSSN	STR9	no	yes	EMPLOYEE
EMPLOYEE	DNO	INTEGER	no	yes	DEPARTMENT
DEPARTMENT	DNAME	VSTR10	no	no	
DEPARTMENT	DNUMBER	INTEGER	yes	no	
DEPARTMENT	MGRSSN	STR9	no	yes	EMPLOYEE
DEPARTMENT	MGRSTARTDATE	STR10	no	no	
DEPT_LOCATIONS	DNUMBER	INTEGER	yes	yes	DEPARTMENT
DEPT_LOCATIONS	DLOCATION	VSTR15	yes	no	
PROJECT	PNAME	VSTR10	no	no	
PROJECT	PNUMBER	INTEGER	yes	no	
PROJECT	PLOCATION	VSTR15	no	no	
PROJECT	DNO	INTEGER	no	yes	DEPARTMENT
WORKS_ON	ESSN	STR9	yes	yes	EMPLOYEE
WORKS_ON	PNO	INTEGER	yes	yes	PROJECT
WORKS_ON	HOURS	REAL	no	no	
DEPENDENT	ESSN	STR9	yes	yes	EMPLOYEE
DEPENDENT	DEPENDENT_NAME	VSTR15	yes	no	
DEPENDENT	SEX	CHAR	no	no	
DEPENDENT	BDATE	STR9	no	no	

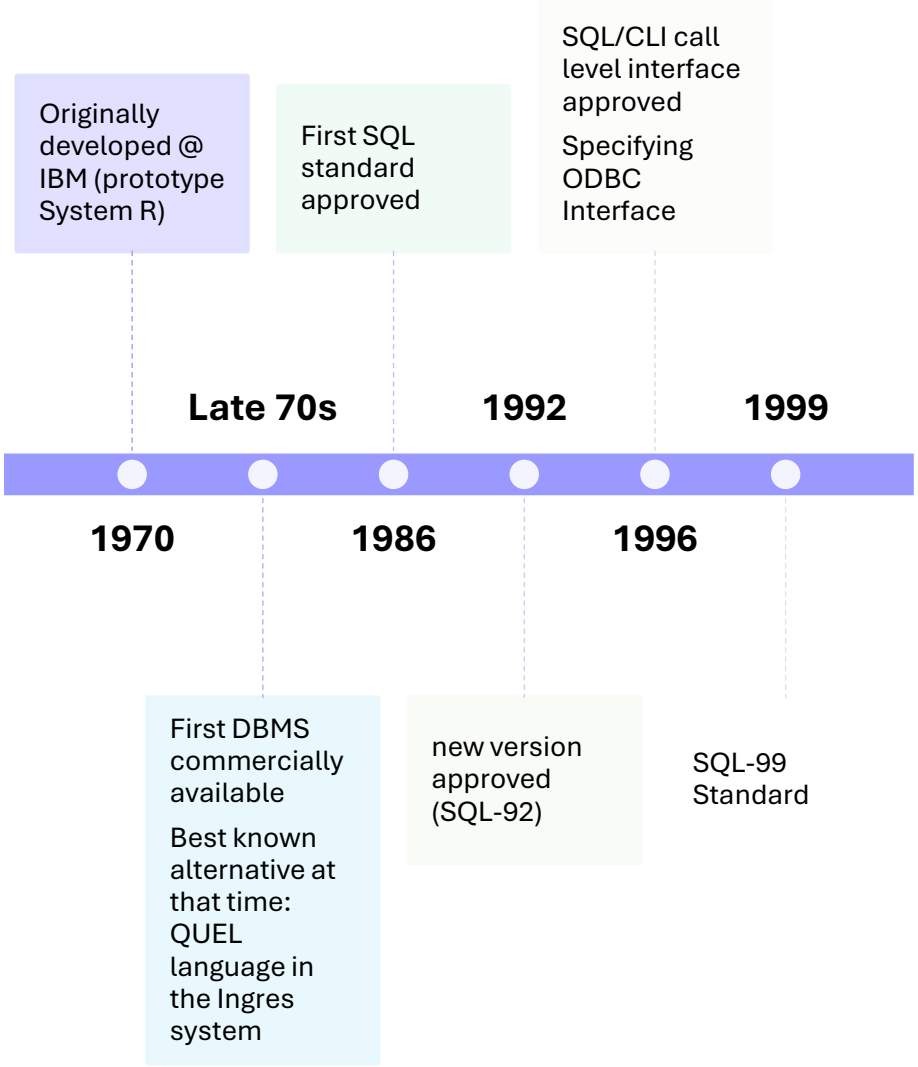
DBMS - Languages

What data language should I learn?

- ✓ **SQL** has traditionally been the most popular DBMS query language



History of SQL



Why use models?

- **Models -** useful to examine / manage part of the real world

- **Cost** of simulation with a model often lower than experimenting with real world

Examples

- Airplane Simulator
- Nuclear Power Plant simulator
- Flood warning system

Data Model

- The **Structure** of the database **includes:**
  - **Data** that will be stored
  - **Relationships**
  - **Constraints** on the data

- **Data Model**
  - Collection of **Concepts** describing **Structure** of database
  - **Data Abstraction** - Hides details of data storage not needed by most users

Categories of Data Models

Categorization of Data Models is done according to the **types of concepts** they use to describe the database structure:

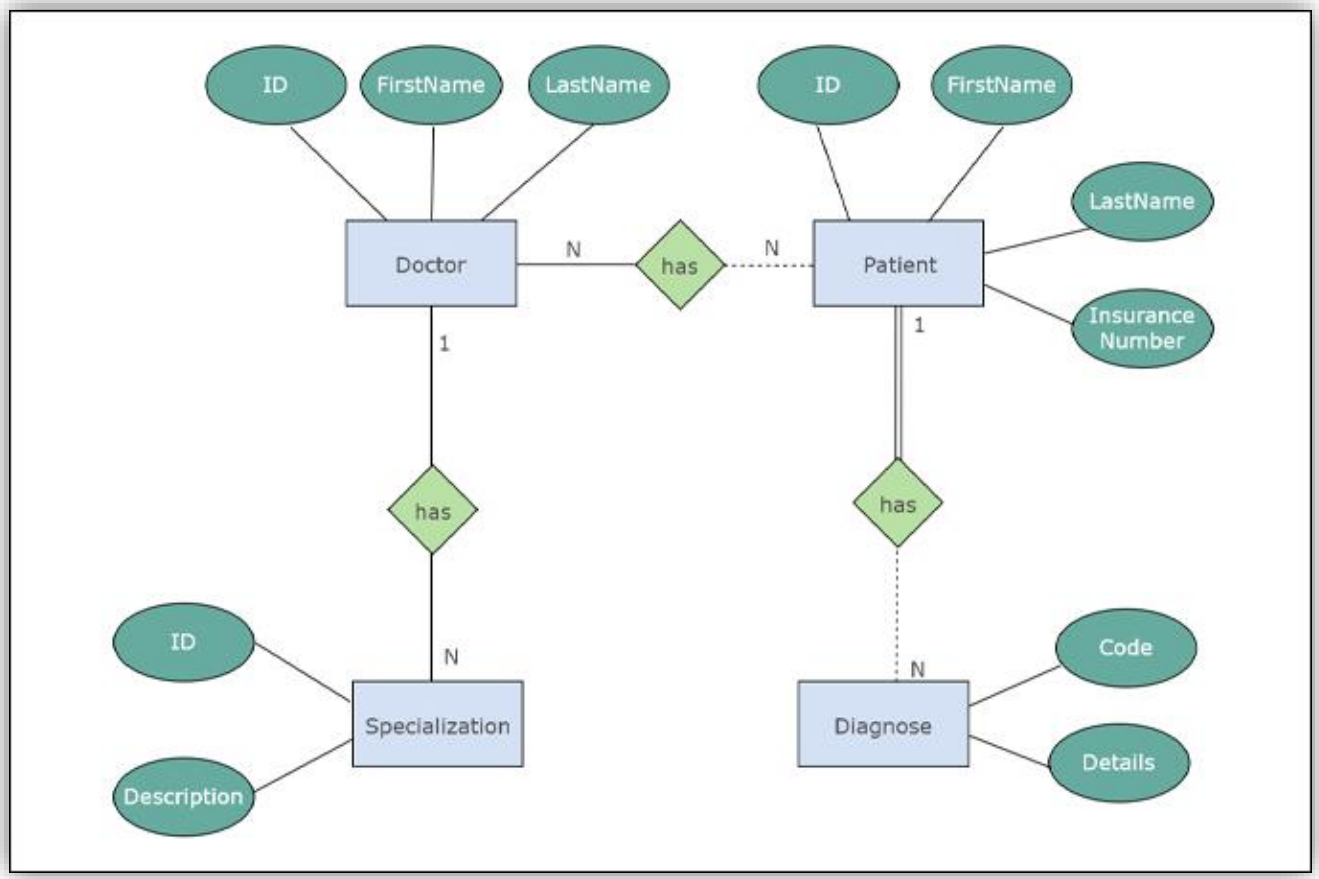
- **Conceptual** or High-Level
- **Implementation** or Representational
- **Physical** or Low-Level

Conceptual Data Models

- Understandable by humans
- Terms
  - **Entity** - represents real-world object / concept / something that exists (physically or conceptually)
  - **Attribute** - represents property describing entity
  - **Relationship** - represents some interaction among two or more entities

✓ **Entity-Relationship Diagram (ERD)**

Conceptual DMs - ERD



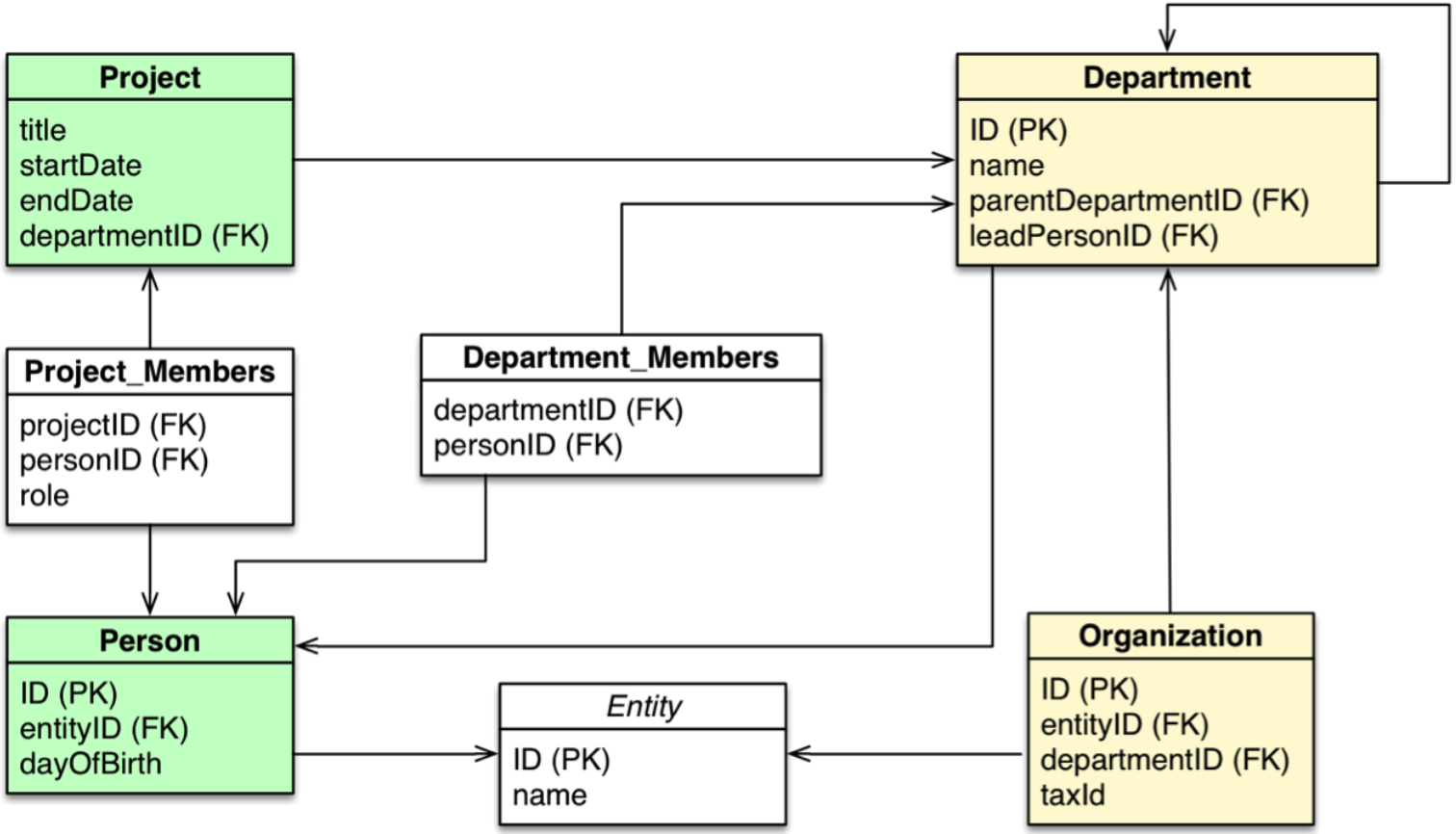
Implementation Data Models

- Understandable by humans and computers
- Directly implemented by computers
  - hide details about storage

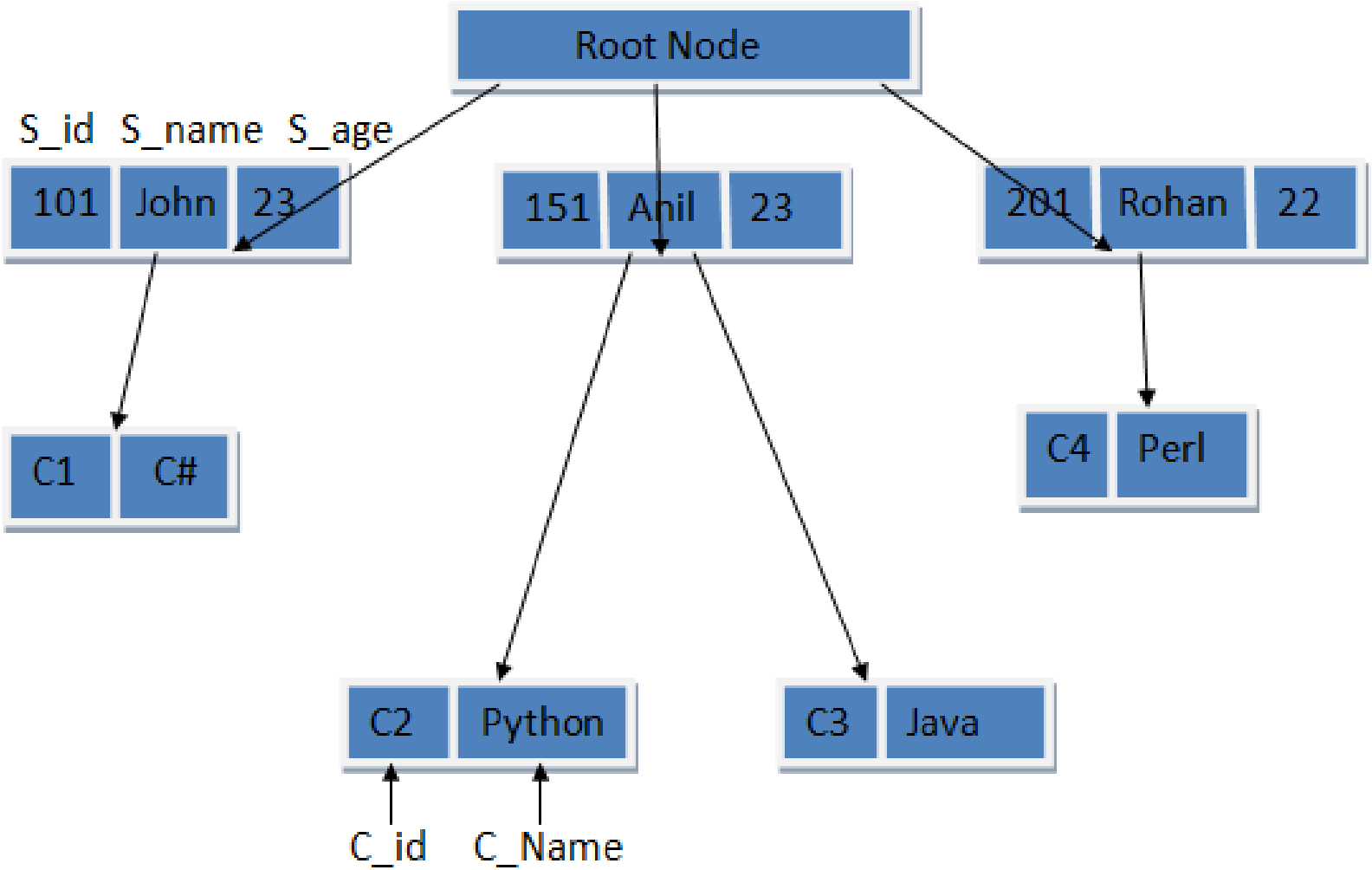
✓ Most frequently used in commercial DBMS
  - **Relational**, Object-Oriented, Object-Relational
  - Network, Hierarchical

➢ How to transform between **Conceptual** and **Implementation?**
  - There are algorithms to do this
  - We will do this in class and in the group project!

Implementation DMs - Relational

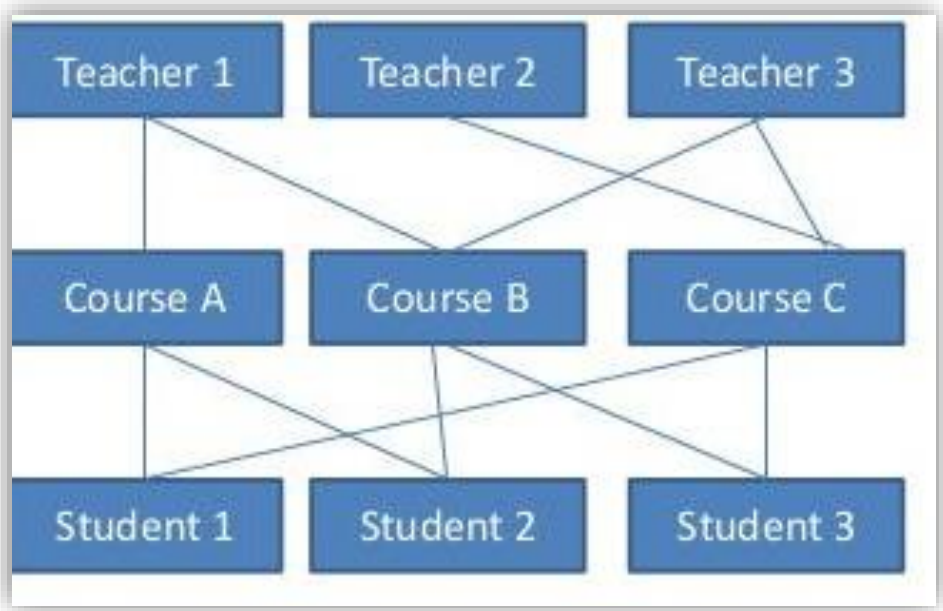


Implementation DMs - Hierarchical



Implementation DMs - Network

- Based on “owner-member relationship”
- Boxes represent records (items)
- Lines represent links (relationships)



Physical DMs

- Understandable by computers and computer specialists

- Describe how data is stored in the computer
  - Record format, Record ordering, Access path

Terminology: Schema vs State

- **Schema** = the **structure** of the database
  - Database blueprint (Design)
  - Tables, columns, relationships
  - Static - rarely changes
  - Can be represented graphically

- **Instance** = the actual data (State) at a particular moment in time
  - Many instances may correspond to one schema
  - An instance can be backed-up and restored later

Schema Example

STUDENT

SNumber	Name	Class	Major

COURSE

CNumber	Name	Credits	Department

PREREQUISITE

CourseNumber	PrereqNumber

SECTION

SIdentifier	CourseNumber	Semester	Year	Instructor

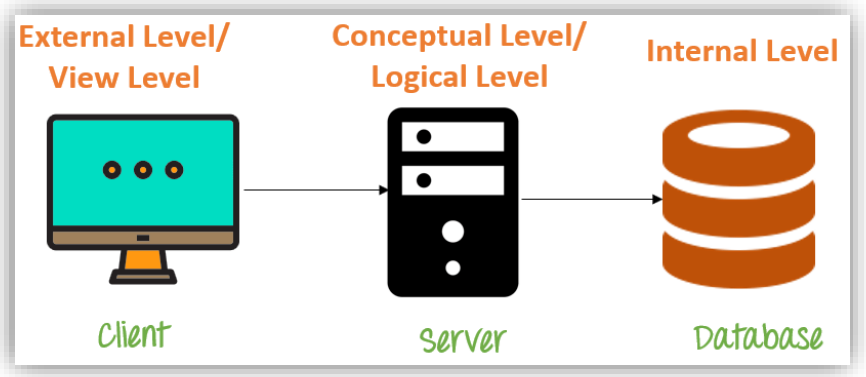
GRADE_REPORT

StudentNumber	SectionIdentifier	Grade

Three-Schema Architecture

Concept: Separate user views from physical storage

- **External** (View) - *What Users see*
  - Multiple customized views for different user groups
- **Conceptual** (Logical) - *Complete Structure*
  - Entire database design with all the entities and relationships
- **Internal** (Physical) - *Storage Details*
  - How data is physically stored on disk with indexes



Data Independence

- ✓ Change one level without affecting others
  - Physical
    - Change storage structure
    - Add / remove indexes
  - Logical
    - Modify design, e.g add new tables / columns

- **Key Benefits**
  - **Security** - users what they need
  - **Flexibility** - changes isolated to one level
  - **Maintainability** - easier system evolution
  - **Multiple user views** from single database

Data Independence

Example - University Database

- **External:** Students see grades, faculty see rosters
- **Conceptual:** Students, Courses, Enrollment tables
- **Internal:** B-tree indexes, hash files, SSD Storage
- **Independence:** Add index without changing user views

Next…

- Lab #1
  - Familiarize with technology and tools